



RTL Design Sherpa

RTL AMBA ACE Module Reference — Rev 0.90

October 2026

Table of Contents

1 ACE (AXI Coherency Extensions) Modules.....	7
1.1 Overview.....	7
1.1.1 What is here.....	7
1.1.2 Key Features.....	7
1.1.3 Module Categories.....	8
1.2 Functional Description.....	9
1.2.1 ACE adds to AXI4.....	9
1.2.2 Snoop channels.....	10
1.2.3 Ordering rules.....	11
1.2.4 CRRESP bit semantics.....	11
1.3 Timing.....	11
1.3.1 Throughput.....	11
1.3.2 Latency.....	12
1.4 Usage Example.....	12
1.4.1 ACE read master.....	12
1.4.2 Cache-side snoop responder.....	13
1.5 Design Notes.....	14
1.5.1 Buffer depth guidelines.....	14
1.5.2 Snoop monitor vs. front-side monitor.....	14
1.5.3 What is not here.....	14
1.6 Related Modules.....	14
1.7 References.....	14

1.8 Navigation.....	15
2 axi4ace_master_rd.....	16
2.1 Overview.....	16
2.2 Parameters.....	16
2.3 Ports.....	17
2.3.1 Module Declaration.....	17
2.3.2 Clock and Reset.....	18
2.3.3 Slave AXI Interface (Input Side - FUB).....	19
2.3.4 Master AXI Interface (Output Side).....	20
2.3.5 ACE Acknowledge.....	22
2.3.6 Status Interface.....	22
2.4 Functional Description.....	22
2.4.1 Dual Buffer Design.....	22
2.4.2 RACK Generation.....	22
2.4.3 Busy Signal Generation.....	23
2.5 ACE Notes.....	23
2.6 Related Modules.....	23
2.7 Navigation.....	23
3 axi4ace_master_wr.....	24
3.1 Overview.....	24
3.2 Parameters.....	24
3.3 Ports.....	25
3.3.1 Module Declaration.....	25
3.3.2 Clock and Reset.....	27

3.3.3 Slave AXI Interface (Input Side - FUB).....	27
3.3.4 Master AXI Interface (Output Side).....	29
3.3.5 ACE Acknowledge.....	31
3.3.6 Status Interface.....	31
3.4 Functional Description.....	31
3.4.1 Triple Buffer Design.....	31
3.4.2 WACK Generation.....	32
3.4.3 Busy Signal Generation.....	32
3.5 ACE Notes.....	32
3.6 Related Modules.....	32
3.7 Navigation.....	32
4 axi4ace_slave_rd.....	33
4.1 Overview.....	33
4.2 Parameters.....	33
4.3 Ports.....	34
4.3.1 Module Declaration.....	34
4.3.2 Clock and Reset.....	35
4.3.3 Slave AXI Interface (Input Side).....	36
4.3.4 Master AXI Interface (Output Side - FUB).....	37
4.3.5 Status Interface.....	39
4.4 Functional Description.....	39
4.4.1 Dual Buffer Design.....	39
4.4.2 Busy Signal Generation.....	39
4.5 ACE Notes.....	39

4.6 Related Modules.....	40
4.7 Navigation.....	40
5 axi4ace_slave_wr.....	41
5.1 Overview.....	41
5.2 Parameters.....	41
5.3 Ports.....	42
5.3.1 Module Declaration.....	42
5.3.2 Clock and Reset.....	44
5.3.3 Slave AXI Interface (Input Side).....	44
5.3.4 Master AXI Interface (Output Side - FUB).....	46
5.3.5 Status Interface.....	48
5.4 Functional Description.....	48
5.4.1 Triple Buffer Design.....	48
5.4.2 Busy Signal Generation.....	48
5.5 ACE Notes.....	49
5.6 Related Modules.....	49
5.7 Navigation.....	49
6 axi4ace_snoop_master.....	50
6.1 Overview.....	50
6.2 Parameters.....	50
6.3 Ports.....	51
6.3.1 Module Declaration.....	51
6.3.2 Clock and Reset.....	52
6.3.3 CCU Interface (Input Side - FUB).....	52

6.3.4	Cache Responder Interface (Output Side - AXI).....	53
6.3.5	Status Interface.....	54
6.4	Functional Description.....	54
6.4.1	Triple Buffer Design.....	54
6.4.2	Busy Signal Generation.....	54
6.5	ACE Notes.....	55
6.6	Related Modules.....	55
6.7	Navigation.....	55
7	axi4ace_snoop_slave.....	56
7.1	Overview.....	56
7.2	Parameters.....	56
7.3	Ports.....	57
7.3.1	Module Declaration.....	57
7.3.2	Clock and Reset.....	58
7.3.3	Manager Interface (Input Side - AXI).....	58
7.3.4	Cache FSM Interface (Output Side - FUB).....	59
7.3.5	Status Interface.....	60
7.4	Functional Description.....	60
7.4.1	Triple Buffer Design.....	60
7.4.2	Busy Signal Generation.....	61
7.5	ACE Notes.....	61
7.6	Related Modules.....	61
7.7	Navigation.....	61

1 ACE (AXI Coherency Extensions) Modules

Location: rtl/amba/ace/ **Status:** Production Ready

1.1 Overview

The ACE subsystem is the AMBA AXI Coherency Extensions transport layer for the cache-ip family (amber/jet caches, onyx CCU). ACE extends AXI4 with coherent transaction-type fields on the address channels (ARSN00P[3:0], AWSN00P[2:0]) and three additional snoop channels (AC/CR/CD) so that a coherency manager can broadcast snoops to peer caches and gather their responses.

These modules are **channel movers with skid buffers**, not protocol controllers. They carry the ACE-shaped port contract defined in projects/components/cache-ip/References/AMBA_ACE_Interface_Definition.md: the onyx D2 subset supports ReadShared, ReadUnique, CleanUnique, MakeUnique, WriteBack, and Evict; full ACE/ACE-Lite conformance is intentionally not claimed.

1.1.1 What is here

- Six base channel movers (base + _monlite only; no _cg, _mon, or stubs):
 - Front-side read/write masters and slaves (axi4ace_master_rd/wr, axi4ace_slave_rd/wr)
 - Snoop-side cache responder and CCU initiator (axi4ace_snoop_slave/master)
- Eight _monlite wrappers in rtl/amba/ace/ that wrap the base movers with the lite-monitor discipline.
- One new ACE-aware monitor core in rtl/amba/monitor/axi4ace_snoop_monitor_lite.sv that tracks snoops in AC-issue order because snoop channels have no ID.

1.1.2 Key Features

- **AXI4 + ACE payloads:** Full AXI4 address/data channels plus ARSN00P/AWSN00P
- **Auto-pulsed acknowledges:** Master read/write movers generate RACK/WACK one cycle after the transaction completes (no ordering semantics in the current subset)

- **Snoop transport:** AC in, CR/CD out (cache side) or AC out, CR/CD in (CCU side)
- **Lite monitoring:** Front-side wrappers use `axi_monitor_lite`; snoop-side wrappers use `axi4ace_snoop_monitor_lite`
- **Skid-buffer elasticity:** Per-channel configurable depth, same `gaxi_skid_buffer` infrastructure as the rest of the AMBA library

1.1.3 Module Categories

1.1.3.1 Core Front-Side Movers

Module	Description	Documentation	Status
axi4ace_master_rd	ACE read master with ARSN00P[3:0] and auto-pulsed RACK	axi4ace_master_rd.md	Documented
axi4ace_master_wr	ACE write master with AWSN00P[2:0] and auto-pulsed WACK	axi4ace_master_wr.md	Documented
axi4ace_slave_rd	ACE read slave with ARSN00P[3:0] pass-through	axi4ace_slave_rd.md	Documented
axi4ace_slave_wr	ACE write slave with AWSN00P[2:0] pass-through	axi4ace_slave_wr.md	Documented

1.1.3.2 Snoop-Side Movers

Module	Description	Documentation	Status
axi4ace_snoop_slave	Cache-side snoop responder transport: AC in, CR/CD out	axi4ace_snoop_slave.md	Documented
axi4ace_snoop_master	CCU-side snoop initiator transport: AC out, CR/CD in	axi4ace_snoop_master.md	Documented

1.1.3.3 Monitor Wrappers

All eight wrappers are documented collectively in [monitor/axi_monitor_lite_wrappers.md](#).

Wrapper	Core	Monitor core	Role
axi4ace_master_rd_monlite	axi4ace_master_rd	axi_monitor_lite	Front-side read master + lite monitor
axi4ace_master_wr_monlite	axi4ace_master_wr	axi_monitor_lite	Front-side write master + lite monitor
axi4ace_slave_rd_monlite	axi4ace_slave_rd	axi_monitor_lite	Front-side read slave + lite monitor
axi4ace_slave_wr_monlite	axi4ace_slave_wr	axi_monitor_lite	Front-side write slave + lite monitor
axi4ace_snoop_slave_monlite	axi4ace_snoop_slave	axi4ace_snoop_monitor_lite	Cache-side snoop responder + snoop monitor
axi4ace_snoop_master_monlite	axi4ace_snoop_master	axi4ace_snoop_monitor_lite	CCU-side snoop initiator + snoop monitor

The front-side _monlite wrappers use the same axi_monitor_lite taps as the AXI4/AXI5 families; the snoop-side wrappers use the new axi4ace_snoop_monitor_lite core because snoop channels have no transaction ID.

1.2 Functional Description

1.2.1 ACE adds to AXI4

On the front side, every address channel gains a transaction-type field:

Channel	Field	Width	Meaning
AR	ARSN00P	4	Read-side coherent transaction type
AW	AWSN00P	3	Write-side coherent transaction type

Data and response channels are unchanged AXI4. Masters additionally drive:

Signal	Direction	Meaning
RACK	Output	One-cycle pulse after the last read data beat
WACK	Output	One-cycle pulse after the write response

The current subset gives these acknowledges no ordering semantics; they are auto-pulsed promptly by the master movers and recorded as such.

1.2.2 Snoop channels

The snoop side uses three channels between the coherency manager (onyx) and each peer cache (amber/jet):

Channel	Dir (cache view)	Payload	Meaning
AC	In	ACADDR, ACSN00P[3:0], ACPR0T[2:0]	Snoop address/command
CR	Out	CRRESP[4:0]	Snoop response
CD	Out	CDDATA, CDLAST	Snoop data

The cache-side mover is `axi4ace_snoop_slave`; the manager-side mover is `axi4ace_snoop_master`. Both are pure transports: they buffer the channels and present the same payload on the FUB side as on the AXI side.

1.2.3 Ordering rules

ACE requires:

- CR and CD only after the AC handshake
- CR responses in the same order as AC addresses
- CD data in the same order as AC addresses
- Multiple outstanding snoops are allowed

The cache-ip family convention is stricter: a cache completes all CD beats for a snoop before asserting its CR, so onyx sees one fully-closed response at a time. That convention is enforced by the cache adapter, not by these transport modules.

1.2.4 CRRESP bit semantics

Bit	Name	Meaning when set
[0]	DataTransfer	Responder will drive data on CD
[1]	Error	Responder's lookup had an error
[2]	PassDirty	Dirty responsibility moves with the data
[3]	IsShared	Another master may hold a copy
[4]	WasUnique	Line was unique in the responder (IHI 0022)

1.3 Timing

1.3.1 Throughput

Configuration	Address CH	Data CH	Notes
Single transaction	1 txn/cycle	1 beat/cycle	When buffers not stalled
Burst (len=16)	1 txn/16 cycles	1 beat/cycle	Sustained
Multiple outstanding	Up to table depth	1 beat/cycle	Out-of-order on

Configuration	Address CH	Data CH	Notes
			front side, in-order on snoop side

1.3.2 Latency

Path	Cycles	Notes
AR → R (single)	2-3	Minimum read latency
AW → B (single)	2-3	Minimum write latency
AC → CR (single)	2-3	Snoop response latency
Buffer overhead	+1 per channel	Skid buffer latency

1.4 Usage Example

1.4.1 ACE read master

```

axi4ace_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_ace_rd_master (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),
    // FUB side (upstream requestor)
    .fub_axi_arid   (cpu_arid),
    .fub_axi_araddr (cpu_araddr),
    .fub_axi_arlen  (cpu_arlen),
    .fub_axi_arsnoop(cpu_arsnoop), // ACE read transaction type
    // ...
    // Master side (downstream manager/CCU)
    .m_axi_arid     (m_arid),
    .m_axi_araddr   (m_araddr),
    .m_axi_arsnoop  (m_arsnoop),
    // ...
    .m_axi_rack     (m_rack), // auto-pulsed by the module
    .busy           (rd_busy)
);

```

1.4.2 Cache-side snoop responder

```
axi4ace_snoop_slave #(
    .SKID_DEPTH_AC(2),
    .SKID_DEPTH_CR(4),
    .SKID_DEPTH_CD(4),
    .ADDR_WIDTH(32),
    .DATA_WIDTH(64)
) u_snoop_slave (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),
    // Manager/CCU side
    .m_axi_acaddr  (snp_acaddr),
    .m_axi_acsnoop (snp_acsnoop),
    .m_axi_acprot  (snp_acprot),
    .m_axi_acvalid (snp_acvalid),
    .m_axi_acready (snp_acready),
    .m_axi_crresp  (snp_crresp),
    .m_axi_crvalid (snp_crvalid),
    .m_axi_crready (snp_crready),
    .m_axi_cddata  (snp_cddata),
    .m_axi_cdlast  (snp_cdlast),
    .m_axi_cdvalid (snp_cdvalid),
    .m_axi_cdready (snp_cdready),
    // Cache FSM side
    .fub_acaddr    (fub_acaddr),
    .fub_acsnoop   (fub_acsnoop),
    .fub_acprot    (fub_acprot),
    .fub_acvalid   (fub_acvalid),
    .fub_acready   (fub_acready),
    .fub_crresp    (fub_crresp),
    .fub_crvalid   (fub_crvalid),
    .fub_crready   (fub_crready),
    .fub_cddata    (fub_cddata),
    .fub_cdlast    (fub_cdlast),
    .fub_cdvalid   (fub_cdvalid),
    .fub_cdready   (fub_cdready),
    .busy          (snp_busy)
);
```

1.5 Design Notes

1.5.1 Buffer depth guidelines

SKID_DEPTH_* is an entry count, not a log2 exponent. Legal values are 2..8 inclusive (any integer, odd depths are legal). Values above 8 fail elaboration; place a `gaxi_fifo_sync` stage ahead of the module for deeper elasticity.

1.5.2 Snoop monitor vs. front-side monitor

The front-side ACE wrappers reuse `axi_monitor_lite`, which tracks AXI transactions by ID. The snoop-side wrappers use `axi4ace_snoop_monitor_lite` because snoop channels have no ID: the monitor keeps a circular table of outstanding snoops in AC-issue order and attributes each CR/CDLAST to the oldest entry.

1.5.3 What is not here

- No `_cg` variants: clock gating is not provided for ACE in this release
 - No `_mon` variants: only the lite-monitor wrappers exist
 - No stubs: these are production channel movers
 - ACE-Lite is not separately implemented; the `cache-ip` contract is full ACE-shaped on the pins documented here
-

1.6 Related Modules

- [axi4ace_snoop_monitor_lite](#) — ACE snoop-channel monitor core
 - [axi_monitor_lite_wrappers](#) — the eight `_monlite` wrappers
 - [AXI4 modules](#) — the base protocol these modules extend
 - [AMBA ACE Interface Definition](#) — the in-repo port contract
-

1.7 References

- ARM IHI 0022H: *AMBA AXI and ACE Protocol Specification* (ACE chapters) — cited, not mirrored
 - `projects/components/cache-ip/References/AMBA_ACE_Interface_Definition.md` — working interface definition for this family
-

Last Updated: 2026-10-05

1.8 Navigation

- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

2 axi4ace_master_rd

An AXI4 + ACE read master module that buffers read address and data channels while carrying the ACE ARSN00P[3:0] transaction-type field and auto-pulsing the RACK acknowledge.

2.1 Overview

The axi4ace_master_rd module is the ACE extension of axi4_master_rd. It provides the same dual skid-buffer architecture for the AR and R channels, adds ARSN00P[3:0] to the address-channel payload, and generates a one-cycle RACK pulse after the last read data beat. It is a transport module: it does not interpret snoop encodings or maintain coherency state.

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	$IW+AW+8+3+2+1+4+3+4+4+UW+4$	Packed AR skid payload width
RSize	int	$IW+DW+2+1+UW$	Packed R skid payload width

2.3 Ports

2.3.1 Module Declaration

```
module axi4ace_master_rd #(
    parameter int SKID_DEPTH_AR      = 2,
    parameter int SKID_DEPTH_R       = 4,
    parameter int AXI_ID_WIDTH       = 8,
    parameter int AXI_ADDR_WIDTH     = 32,
    parameter int AXI_DATA_WIDTH     = 32,
    parameter int AXI_USER_WIDTH     = 1,
    parameter int AW                 = AXI_ADDR_WIDTH,
    parameter int DW                 = AXI_DATA_WIDTH,
    parameter int IW                 = AXI_ID_WIDTH,
    parameter int UW                 = AXI_USER_WIDTH,
    parameter int ARSize             = IW+AW+8+3+2+1+4+3+4+4+UW+4,
    parameter int RSize              = IW+DW+2+1+UW
) (
    // Global Clock and Reset
    input logic                                aclk,
    input logic                                aresetn,

    // Slave AXI Interface (Input Side)
    // Read address channel (AR)
    input logic [IW-1:0]                      fub_axi_arid,
    input logic [AW-1:0]                      fub_axi_araddr,
    input logic [7:0]                         fub_axi_arlen,
    input logic [2:0]                         fub_axi_arsize,
    input logic [1:0]                         fub_axi_arburst,
    input logic                               fub_axi_arlock,
    input logic [3:0]                         fub_axi_arcache,
    input logic [2:0]                         fub_axi_arprot,
    input logic [3:0]                         fub_axi_arqos,
    input logic [3:0]                         fub_axi_arregion,
    input logic [UW-1:0]                     fub_axi_aruser,
    input logic [3:0]                         fub_axi_arsnoop,
    input logic                               fub_axi_arvalid,
    output logic                              fub_axi_arready,

    // Read data channel (R)
    output logic [IW-1:0]                     fub_axi_rid,
    output logic [DW-1:0]                     fub_axi_rdata,
    output logic [1:0]                       fub_axi_rresp,
    output logic                             fub_axi_rlast,
    output logic [UW-1:0]                     fub_axi_ruser,
    output logic                             fub_axi_rvalid,
    input logic                              fub_axi_rready,
```

```

// Master AXI Interface (Output Side)
// Read address channel (AR)
output logic [IW-1:0]          m_axi_arid,
output logic [AW-1:0]          m_axi_araddr,
output logic [7:0]             m_axi_arlen,
output logic [2:0]             m_axi_arsize,
output logic [1:0]             m_axi_arburst,
output logic                   m_axi_arlock,
output logic [3:0]             m_axi_arcache,
output logic [2:0]             m_axi_arprot,
output logic [3:0]             m_axi_arqos,
output logic [3:0]             m_axi_arregion,
output logic [UW-1:0]          m_axi_aruser,
output logic [3:0]             m_axi_arsnoop,
output logic                   m_axi_arvalid,
input  logic                   m_axi_arready,

// Read data channel (R)
input  logic [IW-1:0]          m_axi_rid,
input  logic [DW-1:0]          m_axi_rdata,
input  logic [1:0]             m_axi_rresp,
input  logic                   m_axi_rlast,
input  logic [UW-1:0]          m_axi_ruser,
input  logic                   m_axi_rvalid,
output logic                   m_axi_rready,

// ACE read acknowledge
output logic                   m_axi_rack,

// Status outputs for clock gating
output logic                   busy
);

```

2.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

2.3.3 Slave AXI Interface (Input Side - FUB)

2.3.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Input	Read address

Port	Width	Direction	Description
			ID
fub_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
fub_axi_arlen	8	Input	Burst length, AXI-encoded: beats - 1
fub_axi_arsize	3	Input	Transfer size (bytes per beat)
fub_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Input	Lock type (atomic access)
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_arregion	4	Input	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_arsnoop	4	Input	ACE read transaction type
fub_axi_arvalid	1	Input	Read address valid
fub_axi_arready	1	Output	Read address ready

2.3.3.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WIDTH	Output	Read data ID
fub_axi_rdata	AXI_DATA_WIDTH	Output	Read data
fub_axi_resp	2	Output	Read response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_rlastst	1	Output	Last read transfer in burst
fub_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

2.3.4 Master AXI Interface (Output Side)

2.3.4.1 Read Address Channel (AR)

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length, AXI-encoded: beats - 1
m_axi_arsize	3	Output	Transfer size (bytes per beat)
m_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)

Port	Width	Direction	Description
m_axi_arlock	1	Output	Lock type (atomic access)
m_axi_arcache	4	Output	Cache attributes
m_axi_arprot	3	Output	Protection attributes
m_axi_arqos	4	Output	Quality of Service
m_axi_arregion	4	Output	Region identifier
m_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_arsnoop	4	Output	ACE read transaction type
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready

2.3.4.2 Read Data Channel (R)

Port	Width	Direction	Description
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	2	Input	Read response (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	1	Input	Last read transfer in burst
m_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
m_axi_rval	1	Input	Read data valid

Port	Width	Direction	Description
id			
m_axi_rready	1	Output	Read data ready

2.3.5 ACE Acknowledge

Port	Width	Direction	Description
m_axi_rack	1	Output	One-cycle pulse one cycle after the last read data beat handshake

2.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

2.4 Functional Description

2.4.1 Dual Buffer Design

The module employs the same dual skid buffer architecture as axi4_master_rd:

1. **AR Channel Buffer:** Buffers read address transactions, including ARSN00P
2. **R Channel Buffer:** Buffers read data responses

2.4.2 RACK Generation

The ACE read acknowledge is auto-pulsed by the module:

```
assign m_axi_rack = r_rack;
// r_rack <= m_axi_rvalid && m_axi_rready && m_axi_rlast;
```

A one-cycle pulse is emitted one cycle after the last read data beat handshake. The current subset does not rely on RACK for ordering; downstream consumers that need ordering semantics must implement them externally.

2.4.3 Busy Signal Generation

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
              fub_axi_arvalid || m_axi_rvalid;
```

2.5 ACE Notes

- ARSN00P[3:0] is a straight payload pass-through; the module does not decode or validate the encoding
- RACK is generated locally; do not drive m_axi_rack from outside
- IDs (ARID/RID) are carried end-to-end; the module performs no ID matching or reordering
- The module is part of the ACE-shaped port contract for the cache-ip family; see the contract doc for transaction-type semantics

2.6 Related Modules

- **axi4ace_master_rd_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_slave_rd**: Complementary ACE read slave
 - **axi4_master_rd**: AXI4-only base module
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

2.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

3 axi4ace_master_wr

An AXI4 + ACE write master module that buffers AW, W, and B channels while carrying the ACE AWSN00P[2:0] transaction-type field and auto-pulsing the WACK acknowledge.

3.1 Overview

The axi4ace_master_wr module is the ACE extension of axi4_master_wr. It provides skid-buffer elasticity for the AW, W, and B channels, adds AWSN00P[2:0] to the write address payload, and generates a one-cycle WACK pulse after the write response handshake. It is a transport module and does not interpret snoop encodings.

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Write address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_W	int	4	Write data channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_B	int	2	Write response channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width

Parameter	Type	Default	Description
UW	int	AXI_USER_WIDT H	Short alias for user width
AWSize	int	IW+AW+8+3+2+ 1+4+3+4+4+UW+ 3	Packed AW skid payload width
WSize	int	DW+SW+1+UW	Packed W skid payload width
BSize	int	IW+2+UW	Packed B skid payload width

3.3 Ports

3.3.1 Module Declaration

```

module axi4ace_master_wr #(
    parameter int SKID_DEPTH_AW      = 2,
    parameter int SKID_DEPTH_W       = 4,
    parameter int SKID_DEPTH_B       = 2,
    parameter int AXI_ID_WIDTH       = 8,
    parameter int AXI_ADDR_WIDTH     = 32,
    parameter int AXI_DATA_WIDTH     = 32,
    parameter int AXI_USER_WIDTH     = 1,
    parameter int AXI_WSTRB_WIDTH    = AXI_DATA_WIDTH / 8,
    parameter int AW                  = AXI_ADDR_WIDTH,
    parameter int DW                  = AXI_DATA_WIDTH,
    parameter int IW                  = AXI_ID_WIDTH,
    parameter int SW                  = AXI_WSTRB_WIDTH,
    parameter int UW                  = AXI_USER_WIDTH,
    parameter int ASize               = IW+AW+8+3+2+1+4+3+4+4+UW+3,
    parameter int WSize               = DW+SW+1+UW,
    parameter int BSize               = IW+2+UW
) (
    // Global Clock and Reset
    input logic                               aclk,
    input logic                               aresetn,

    // Slave AXI Interface (Input Side)
    // Write address channel (AW)
    input logic [IW-1:0]                     fub_axi_awid,
    input logic [AW-1:0]                     fub_axi_awaddr,
    input logic [7:0]                         fub_axi_awlen,
    input logic [2:0]                         fub_axi_awsz,
    input logic [1:0]                         fub_axi_awburst,

```

```

input  logic
input  logic [3:0]
input  logic [2:0]
input  logic [3:0]
input  logic [3:0]
input  logic [UW-1:0]
input  logic [2:0]
input  logic
output logic

// Write data channel (W)
input  logic [DW-1:0]
input  logic [SW-1:0]
input  logic
input  logic [UW-1:0]
input  logic
output logic

// Write response channel (B)
output logic [IW-1:0]
output logic [1:0]
output logic [UW-1:0]
output logic
input  logic

// Master AXI Interface (Output Side)
// Write address channel (AW)
output logic [IW-1:0]
output logic [AW-1:0]
output logic [7:0]
output logic [2:0]
output logic [1:0]
output logic
output logic [3:0]
output logic [2:0]
output logic [3:0]
output logic [3:0]
output logic [UW-1:0]
output logic [2:0]
output logic
input  logic

// Write data channel (W)
output logic [DW-1:0]
output logic [SW-1:0]
output logic
output logic [UW-1:0]

fub_axi_awlock,
fub_axi_awcache,
fub_axi_awprot,
fub_axi_awqos,
fub_axi_awregion,
fub_axi_awuser,
fub_axi_awsnoop,
fub_axi_awvalid,
fub_axi_awready,

fub_axi_wdata,
fub_axi_wstrb,
fub_axi_wlast,
fub_axi_wuser,
fub_axi_wvalid,
fub_axi_wready,

fub_axi_bid,
fub_axi_bresp,
fub_axi_buser,
fub_axi_bvalid,
fub_axi_bready,

m_axi_awid,
m_axi_awaddr,
m_axi_awlen,
m_axi_awsz,
m_axi_awburst,
m_axi_awlock,
m_axi_awcache,
m_axi_awprot,
m_axi_awqos,
m_axi_awregion,
m_axi_awuser,
m_axi_awsnoop,
m_axi_awvalid,
m_axi_awready,

m_axi_wdata,
m_axi_wstrb,
m_axi_wlast,
m_axi_wuser,

```

```

    output logic m_axi_wvalid,
    input logic m_axi_wready,

    // Write response channel (B)
    input logic [IW-1:0] m_axi_bid,
    input logic [1:0] m_axi_bresp,
    input logic [UW-1:0] m_axi_buser,
    input logic m_axi_bvalid,
    output logic m_axi_bready,

    // ACE write acknowledge
    output logic m_axi_wack,

    // Status outputs for clock gating
    output logic busy
);

```

3.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

3.3.3 Slave AXI Interface (Input Side - FUB)

3.3.3.1 Write Address Channel (AW)

Port	Width	Direction	Description
fub_axi_awid	AXI_ID_WIDTH	Input	Write address ID
fub_axi_awaddr	AXI_ADDR_WIDTH	Input	Write address
fub_axi_awlen	8	Input	Burst length, AXI-encoded: beats - 1
fub_axi_awsz	3	Input	Transfer size (bytes per beat)
fub_axi_awburst	2	Input	Burst type (FIXED, INCR, WRAP)

Port	Width	Direction	Description
fub_axi_awlock	1	Input	Lock type (atomic access)
fub_axi_awcache	4	Input	Cache attributes
fub_axi_awprot	3	Input	Protection attributes
fub_axi_awqos	4	Input	Quality of Service
fub_axi_awregion	4	Input	Region identifier
fub_axi_awuser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_awsnoop	3	Input	ACE write transaction type
fub_axi_awvalid	1	Input	Write address valid
fub_axi_awready	1	Output	Write address ready

3.3.3.2 Write Data Channel (W)

Port	Width	Direction	Description
fub_axi_wdata	AXI_DATA_WIDTH	Input	Write data
fub_axi_wstrb	AXI_WSTRB_WIDTH	Input	Write strobes
fub_axi_wlast	1	Input	Last write transfer in burst
fub_axi_wuser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_wvalid	1	Input	Write data valid
fub_axi_wread	1	Output	Write data

Port	Width	Direction	Description
y			ready

3.3.3.3 Write Response Channel (B)

Port	Width	Direction	Description
fub_axi_bid	AXI_ID_WIDTH	Output	Write response ID
fub_axi_bresp	2	Output	Write response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_buser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

3.3.4 Master AXI Interface (Output Side)

3.3.4.1 Write Address Channel (AW)

Port	Width	Direction	Description
m_axi_awid	AXI_ID_WIDTH	Output	Write address ID
m_axi_awaddr	AXI_ADDR_WIDTH	Output	Write address
m_axi_awlen	8	Output	Burst length, AXI-encoded: beats - 1
m_axi_awsz	3	Output	Transfer size (bytes per beat)
m_axi_awburst	2	Output	Burst type (FIXED, INCR, WRAP)
m_axi_awlock	1	Output	Lock type (atomic access)

Port	Width	Direction	Description
m_axi_awcache	4	Output	Cache attributes
m_axi_awprot	3	Output	Protection attributes
m_axi_awqos	4	Output	Quality of Service
m_axi_awregion	4	Output	Region identifier
m_axi_awuser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_awsnoop	3	Output	ACE write transaction type
m_axi_awvalid	1	Output	Write address valid
m_axi_awready	1	Input	Write address ready

3.3.4.2 Write Data Channel (W)

Port	Width	Direction	Description
m_axi_wdata	AXI_DATA_WIDTH	Output	Write data
m_axi_wstrb	AXI_WSTRB_WIDTH	Output	Write strobes
m_axi_wlast	1	Output	Last write transfer in burst
m_axi_wuser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_wvalid	1	Output	Write data valid
m_axi_wready	1	Input	Write data ready

3.3.4.3 Write Response Channel (B)

Port	Width	Direction	Description
m_axi_bid	AXI_ID_WID TH	Input	Write response ID
m_axi_bresp	2	Input	Write response (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_buser	AXI_USER_W IDTH	Input	User-defined signals
m_axi_bvalid	1	Input	Write response valid
m_axi_bready	1	Output	Write response ready

3.3.5 ACE Acknowledge

Port	Width	Direction	Description
m_axi_wack	1	Output	One-cycle pulse one cycle after the write response handshake

3.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

3.4 Functional Description

3.4.1 Triple Buffer Design

The module buffers all three write channels independently:

1. **AW Channel Buffer:** Carries address, control, and AWSN00P
2. **W Channel Buffer:** Carries data, strobes, and WLAST
3. **B Channel Buffer:** Carries write responses back to the FUB side

3.4.2 WACK Generation

The ACE write acknowledge is auto-pulsed by the module:

```
assign m_axi_wack = r_wack;  
// r_wack <= m_axi_bvalid && m_axi_bready;
```

A one-cycle pulse is emitted one cycle after the write response handshake. The current subset does not rely on WACK for ordering.

3.4.3 Busy Signal Generation

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count  
> 0) ||  
              fub_axi_awvalid || fub_axi_wvalid || m_axi_bvalid;
```

3.5 ACE Notes

- AWSNOOP[2:0] is a straight payload pass-through; the module does not decode the encoding
- WACK is generated locally; do not drive m_axi_wack from outside
- IDs (AWID/BID) are carried end-to-end; the module performs no ID matching
- Write data beats are not reordered; they pass through the W skid buffer in order

3.6 Related Modules

- **axi4ace_master_wr_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_slave_wr**: Complementary ACE write slave
 - **axi4_master_wr**: AXI4-only base module
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

3.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

4 axi4ace_slave_rd

An AXI4 + ACE read slave module that buffers AR and R channels while carrying the ACE ARSN00P[3:0] transaction-type field from the slave interface to the backend.

4.1 Overview

The `axi4ace_slave_rd` module is the ACE extension of `axi4_slave_rd`. It receives AXI4 read requests with the ACE ARSN00P[3:0] field on the `s_axi_*` interface, buffers them, and presents them on the `fub_axi_*` backend interface. It is a transport module: it does not generate read data or interpret snoop encodings.

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused, kept for naming consistency)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width

Parameter	Type	Default	Description
ARSize	int	$IW+AW+8+3+2+1+4+3+4+4+UW+4$	Packed AR skid payload width
RSize	int	$IW+DW+2+1+UW$	Packed R skid payload width

4.3 Ports

4.3.1 Module Declaration

```

module axi4ace_slave_rd #(
    parameter int SKID_DEPTH_AR    = 2,
    parameter int SKID_DEPTH_R     = 4,
    parameter int AXI_ID_WIDTH     = 8,
    parameter int AXI_ADDR_WIDTH   = 32,
    parameter int AXI_DATA_WIDTH   = 32,
    parameter int AXI_USER_WIDTH   = 1,
    parameter int AXI_WSTRB_WIDTH  = AXI_DATA_WIDTH / 8,
    parameter int AW               = AXI_ADDR_WIDTH,
    parameter int DW               = AXI_DATA_WIDTH,
    parameter int IW               = AXI_ID_WIDTH,
    parameter int SW               = AXI_WSTRB_WIDTH,
    parameter int UW               = AXI_USER_WIDTH,
    parameter int ARSize           =  $IW+AW+8+3+2+1+4+3+4+4+UW+4$ ,
    parameter int RSize            =  $IW+DW+2+1+UW$ 
) (
    // Global Clock and Reset
    input  logic aclk,
    input  logic aresetn,

    // Slave AXI Interface (Input Side)
    // Read address channel (AR)
    input  logic [IW-1:0] s_axi_arid,
    input  logic [AW-1:0] s_axi_araddr,
    input  logic [7:0] s_axi_arlen,
    input  logic [2:0] s_axi_arsize,
    input  logic [1:0] s_axi_arburst,
    input  logic s_axi_arlock,
    input  logic [3:0] s_axi_arcache,
    input  logic [2:0] s_axi_arprot,
    input  logic [3:0] s_axi_arqos,
    input  logic [3:0] s_axi_arregion,
    input  logic [UW-1:0] s_axi_aruser,
    input  logic [3:0] s_axi_arsnoop,
    input  logic s_axi_arvalid,

```

```

output logic                                s_axi_arready,

// Read data channel (R)
output logic [IW-1:0]                      s_axi_rid,
output logic [DW-1:0]                      s_axi_rdata,
output logic [1:0]                        s_axi_rresp,
output logic                              s_axi_rlast,
output logic [UW-1:0]                    s_axi_ruser,
output logic                              s_axi_rvalid,
input logic                              s_axi_rready,

// Master AXI Interface (Output Side to memory or backend)
// Read address channel (AR)
output logic [IW-1:0]                      fub_axi_arid,
output logic [AW-1:0]                      fub_axi_araddr,
output logic [7:0]                        fub_axi_arlen,
output logic [2:0]                        fub_axi_arsize,
output logic [1:0]                        fub_axi_arburst,
output logic                              fub_axi_arlock,
output logic                              fub_axi_arcache,
output logic [3:0]                        fub_axi_arprot,
output logic [2:0]                        fub_axi_arqos,
output logic [3:0]                        fub_axi_arregion,
output logic [UW-1:0]                    fub_axi_aruser,
output logic [3:0]                        fub_axi_arsnoop,
output logic                              fub_axi_arvalid,
input logic                              fub_axi_arready,

// Read data channel (R)
input logic [IW-1:0]                      fub_axi_rid,
input logic [DW-1:0]                      fub_axi_rdata,
input logic [1:0]                        fub_axi_rresp,
input logic                              fub_axi_rlast,
input logic [UW-1:0]                    fub_axi_ruser,
input logic                              fub_axi_rvalid,
output logic                              fub_axi_rready,

// Status outputs for clock gating
output logic                              busy
);

```

4.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low

Port	Width	Direction	Description
			reset

4.3.3 Slave AXI Interface (Input Side)

4.3.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
s_axi_arid	AXI_ID_WIDTH	Input	Read address ID
s_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
s_axi_arlen	8	Input	Burst length, AXI-encoded: beats - 1
s_axi_arsize	3	Input	Transfer size (bytes per beat)
s_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
s_axi_arlock	1	Input	Lock type (atomic access)
s_axi_arcache	4	Input	Cache attributes
s_axi_arprot	3	Input	Protection attributes
s_axi_arqos	4	Input	Quality of Service
s_axi_arregion	4	Input	Region identifier
s_axi_aruser	AXI_USER_WIDTH	Input	User-defined signals
s_axi_arsnoop	4	Input	ACE read transaction type

Port	Width	Direction	Description
s_axi_arvalid	1	Input	Read address valid
s_axi_arready	1	Output	Read address ready

4.3.3.2 Read Data Channel (R)

Port	Width	Direction	Description
s_axi_rid	AXI_ID_WIDTH	Output	Read data ID
s_axi_rdata	AXI_DATA_WIDTH	Output	Read data
s_axi_rresp	2	Output	Read response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_rlast	1	Output	Last read transfer in burst
s_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready

4.3.4 Master AXI Interface (Output Side - FUB)

4.3.4.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Output	Read address ID
fub_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
fub_axi_arlen	8	Output	Burst length, AXI-encoded: beats - 1
fub_axi_arsize	3	Output	Transfer size (bytes per

Port	Width	Direction	Description
			beat)
fub_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Output	Lock type (atomic access)
fub_axi_arcache	4	Output	Cache attributes
fub_axi_arprot	3	Output	Protection attributes
fub_axi_arqos	4	Output	Quality of Service
fub_axi_arregion	4	Output	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_arsnoop	4	Output	ACE read transaction type
fub_axi_arvalid	1	Output	Read address valid
fub_axi_arready	1	Input	Read address ready

4.3.4.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WIDTH	Input	Read data ID
fub_axi_rdata	AXI_DATA_WIDTH	Input	Read data
fub_axi_resp	2	Input	Read response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_rlast	1	Input	Last read transfer in burst

Port	Width	Direction	Description
st			
fub_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_rvalid	1	Input	Read data valid
fub_axi_rready	1	Output	Read data ready

4.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

4.4 Functional Description

4.4.1 Dual Buffer Design

The module buffers AR from the slave side to the backend and R from the backend back to the slave side:

1. **AR Channel Buffer:** Receives `s_axi_ar*` and presents `fub_axi_ar*`, including ARSN00P
2. **R Channel Buffer:** Receives `fub_axi_r*` and presents `s_axi_r*`

4.4.2 Busy Signal Generation

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
              s_axi_arvalid || fub_axi_rvalid;
```

4.5 ACE Notes

- ARSN00P[3:0] is carried straight through from `s_axi_arsnoop` to `fub_axi_arsnoop`
- The module does not generate RACK; that handshake is produced by the upstream master
- IDs (ARID/RID) are carried end-to-end; the module performs no ID matching

- AXI_WSTRB_WIDTH is present only for naming consistency with the write slave and is unused

4.6 Related Modules

- **axi4ace_slave_rd_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_master_rd**: Complementary ACE read master
 - **axi4_slave_rd**: AXI4-only base module
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

4.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

5 axi4ace_slave_wr

An AXI4 + ACE write slave module that buffers AW, W, and B channels while carrying the ACE AWSN00P[2:0] transaction-type field from the slave interface to the backend.

5.1 Overview

The axi4ace_slave_wr module is the ACE extension of axi4_slave_wr. It receives AXI4 write requests with the ACE AWSN00P[2:0] field on the s_axi_* interface, buffers them, and presents them on the fub_axi_* backend interface. Responses from the backend are buffered back to the slave interface. It is a transport module and does not interpret snoop encodings.

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Write address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_W	int	4	Write data channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_B	int	2	Write response channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width

Parameter	Type	Default	Description
UW	int	AXI_USER_WIDT H	Short alias for user width
AWSize	int	IW+AW+8+3+2+ 1+4+3+4+4+UW+ 3	Packed AW skid payload width
WSize	int	DW+SW+1+UW	Packed W skid payload width
BSize	int	IW+2+UW	Packed B skid payload width

5.3 Ports

5.3.1 Module Declaration

```

module axi4ace_slave_wr #(
    parameter int SKID_DEPTH_AW      = 2,
    parameter int SKID_DEPTH_W       = 4,
    parameter int SKID_DEPTH_B       = 2,
    parameter int AXI_ID_WIDTH       = 8,
    parameter int AXI_ADDR_WIDTH     = 32,
    parameter int AXI_DATA_WIDTH     = 32,
    parameter int AXI_USER_WIDTH     = 1,
    parameter int AXI_WSTRB_WIDTH    = AXI_DATA_WIDTH / 8,
    parameter int AW                 = AXI_ADDR_WIDTH,
    parameter int DW                 = AXI_DATA_WIDTH,
    parameter int IW                 = AXI_ID_WIDTH,
    parameter int SW                 = AXI_WSTRB_WIDTH,
    parameter int UW                 = AXI_USER_WIDTH,
    parameter int ASize              = IW+AW+8+3+2+1+4+3+4+4+UW+3,
    parameter int WSize              = DW+SW+1+UW,
    parameter int BSize              = IW+2+UW
) (
    // Global Clock and Reset
    input logic aclk,
    input logic aresetn,

    // Slave AXI Interface (Input Side)
    // Write address channel (AW)
    input logic [IW-1:0] s_axi_awid,
    input logic [AW-1:0] s_axi_awaddr,
    input logic [7:0] s_axi_awlen,
    input logic [2:0] s_axi_awsz,
    input logic [1:0] s_axi_awburst,

```

```

input    logic
input    logic [3:0]
input    logic [2:0]
input    logic [3:0]
input    logic [3:0]
input    logic [UW-1:0]
input    logic [2:0]
input    logic
output   logic

s_axi_awlock,
s_axi_awcache,
s_axi_awprot,
s_axi_awqos,
s_axi_awregion,
s_axi_awuser,
s_axi_awsnoop,
s_axi_awvalid,
s_axi_awready,

// Write data channel (W)
input    logic [DW-1:0]
input    logic [SW-1:0]
input    logic
input    logic [UW-1:0]
input    logic
output   logic

s_axi_wdata,
s_axi_wstrb,
s_axi_wlast,
s_axi_wuser,
s_axi_wvalid,
s_axi_wready,

// Write response channel (B)
output   logic [IW-1:0]
output   logic [1:0]
output   logic [UW-1:0]
output   logic
input    logic

s_axi_bid,
s_axi_bresp,
s_axi_buser,
s_axi_bvalid,
s_axi_bready,

// Master AXI Interface (Output Side to memory or backend)
// Write address channel (AW)
output   logic [IW-1:0]
output   logic [AW-1:0]
output   logic [7:0]
output   logic [2:0]
output   logic [1:0]
output   logic
output   logic [3:0]
output   logic [2:0]
output   logic [3:0]
output   logic [3:0]
output   logic [UW-1:0]
output   logic [2:0]
output   logic
input    logic

fub_axi_awid,
fub_axi_awaddr,
fub_axi_awlen,
fub_axi_awsz,
fub_axi_awburst,
fub_axi_awlock,
fub_axi_awcache,
fub_axi_awprot,
fub_axi_awqos,
fub_axi_awregion,
fub_axi_awuser,
fub_axi_awsnoop,
fub_axi_awvalid,
fub_axi_awready,

// Write data channel (W)
output   logic [DW-1:0]
output   logic [SW-1:0]
output   logic
output   logic [UW-1:0]

fub_axi_wdata,
fub_axi_wstrb,
fub_axi_wlast,
fub_axi_wuser,

```

```

    output logic fub_axi_wvalid,
    input logic fub_axi_wready,

    // Write response channel (B)
    input logic [IW-1:0] fub_axi_bid,
    input logic [1:0] fub_axi_bresp,
    input logic [UW-1:0] fub_axi_buser,
    input logic fub_axi_bvalid,
    output logic fub_axi_bready,

    // Status outputs for clock gating
    output logic busy
);

```

5.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

5.3.3 Slave AXI Interface (Input Side)

5.3.3.1 Write Address Channel (AW)

Port	Width	Direction	Description
s_axi_awid	AXI_ID_WIDTH	Input	Write address ID
s_axi_awaddr	AXI_ADDR_WIDTH	Input	Write address
s_axi_awlen	8	Input	Burst length, AXI-encoded: beats - 1
s_axi_awsz	3	Input	Transfer size (bytes per beat)
s_axi_awburst	2	Input	Burst type (FIXED, INCR, WRAP)
s_axi_awlock	1	Input	Lock type (atomic access)

Port	Width	Direction	Description
s_axi_awcache	4	Input	Cache attributes
s_axi_awprot	3	Input	Protection attributes
s_axi_awqos	4	Input	Quality of Service
s_axi_awregion	4	Input	Region identifier
s_axi_awuser	AXI_USER_WIDTH	Input	User-defined signals
s_axi_awsnoop	3	Input	ACE write transaction type
s_axi_awvalid	1	Input	Write address valid
s_axi_awready	1	Output	Write address ready

5.3.3.2 Write Data Channel (W)

Port	Width	Direction	Description
s_axi_wdata	AXI_DATA_WIDTH	Input	Write data
s_axi_wstrb	AXI_WSTRB_WIDTH	Input	Write strobes
s_axi_wlast	1	Input	Last write transfer in burst
s_axi_wuser	AXI_USER_WIDTH	Input	User-defined signals
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready

5.3.3.3 Write Response Channel (B)

Port	Width	Direction	Description
s_axi_bid	AXI_ID_WIDTH	Output	Write response ID
s_axi_bresp	2	Output	Write response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_buser	AXI_USER_WIDTH	Output	User-defined signals
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready

5.3.4 Master AXI Interface (Output Side - FUB)

5.3.4.1 Write Address Channel (AW)

Port	Width	Direction	Description
fub_axi_awid	AXI_ID_WIDTH	Output	Write address ID
fub_axi_awaddr	AXI_ADDR_WIDTH	Output	Write address
fub_axi_awlen	8	Output	Burst length, AXI-encoded: beats - 1
fub_axi_awsz	3	Output	Transfer size (bytes per beat)
fub_axi_awburst	2	Output	Burst type (FIXED, INCR, WRAP)
fub_axi_awlock	1	Output	Lock type (atomic access)
fub_axi_awcache	4	Output	Cache attributes

Port	Width	Direction	Description
fub_axi_awprot	3	Output	Protection attributes
fub_axi_awqos	4	Output	Quality of Service
fub_axi_awregion	4	Output	Region identifier
fub_axi_awuser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_awsnoop	3	Output	ACE write transaction type
fub_axi_awvalid	1	Output	Write address valid
fub_axi_awready	1	Input	Write address ready

5.3.4.2 Write Data Channel (W)

Port	Width	Direction	Description
fub_axi_wdata	AXI_DATA_WIDTH	Output	Write data
fub_axi_wstrb	AXI_WSTRB_WIDTH	Output	Write strobes
fub_axi_wlast	1	Output	Last write transfer in burst
fub_axi_wuser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_wvalid	1	Output	Write data valid
fub_axi_wready	1	Input	Write data ready

5.3.4.3 Write Response Channel (B)

Port	Width	Direction	Description
fub_axi_bid	AXI_ID_WIDTH	Input	Write response ID
fub_axi_bresp	2	Input	Write response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_buser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_bvalid	1	Input	Write response valid
fub_axi_bready	1	Output	Write response ready

5.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

5.4 Functional Description

5.4.1 Triple Buffer Design

The module buffers all three write channels:

1. **AW Channel Buffer:** Receives `s_axi_aw*` and presents `fub_axi_aw*`, including `AWSN00P`
2. **W Channel Buffer:** Receives `s_axi_w*` and presents `fub_axi_w*`
3. **B Channel Buffer:** Receives `fub_axi_b*` and presents `s_axi_b*`

5.4.2 Busy Signal Generation

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count > 0) ||  
              s_axi_awvalid || s_axi_wvalid || fub_axi_bvalid;
```

5.5 ACE Notes

- AWSN00P[2:0] is carried straight through from s_axi_awsnoop to fub_axi_awsnoop
- The module does not generate WACK; that handshake is produced by the upstream master
- IDs (AWID/BID) are carried end-to-end; the module performs no ID matching
- Write data beats are not reordered; they pass through the W skid buffer in order

5.6 Related Modules

- **axi4ace_slave_wr_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_master_wr**: Complementary ACE write master
 - **axi4_slave_wr**: AXI4-only base module
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

5.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

6 axi4ace_snoop_master

ACE snoop-channel initiator transport on the coherency manager (CCU) side: AC out to a peer cache, CR/CD in from that cache.

6.1 Overview

The `axi4ace_snoop_master` module sits between the CCU logic (onyx) and a peer cache responder (amber/jet). It receives a snoop command from the CCU on the FUB-side AC channel, buffers it, and drives it to the cache on the `m_axi_*` interface. The cache's CR and CD responses are buffered back to the CCU. It is a transport module and does not generate snoop commands or interpret responses.

6.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AC	int	2	Snoop address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CR	int	4	Snoop response channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CD	int	4	Snoop data channel skid buffer depth in entries (2..8 inclusive)
ADDR_WIDTH	int	32	Snoop address width
DATA_WIDTH	int	32	Snoop data width
AW	int	ADDR_WIDTH	Short alias for address width
DW	int	DATA_WIDTH	Short alias for data width
ACSize	int	AW + 4 + 3	Packed AC skid payload width
CRSize	int	5	Packed CR skid payload width
CDSize	int	DW + 1	Packed CD skid payload width

6.3 Ports

6.3.1 Module Declaration

```
module axi4ace_snoop_master #(
    parameter int SKID_DEPTH_AC = 2,
    parameter int SKID_DEPTH_CR = 4,
    parameter int SKID_DEPTH_CD = 4,
    parameter int ADDR_WIDTH    = 32,
    parameter int DATA_WIDTH   = 32,
    parameter int AW             = ADDR_WIDTH,
    parameter int DW             = DATA_WIDTH,
    parameter int ACSIZE         = AW + 4 + 3,
    parameter int CRSIZE         = 5,
    parameter int CDSIZE         = DW + 1
) (
    // Global Clock and Reset
    input logic                aclk,
    input logic                aresetn,

    // Slave AXI Interface (Input Side) -- upstream CCU logic
    // Snoop address channel (AC)
    input logic [AW-1:0]      fub_acaddr,
    input logic [3:0]         fub_acsnoop,
    input logic [2:0]         fub_acprot,
    input logic               fub_acvalid,
    output logic              fub_acready,

    // Snoop response channel (CR)
    output logic [4:0]        fub_crresp,
    output logic              fub_crvalid,
    input logic               fub_crready,

    // Snoop data channel (CD)
    output logic [DW-1:0]     fub_cddata,
    output logic              fub_cdlast,
    output logic              fub_cdvalid,
    input logic               fub_cdready,

    // Master AXI Interface (Output Side) -- peer cache responder
    // Snoop address channel (AC)
    output logic [AW-1:0]     m_axi_acaddr,
    output logic [3:0]        m_axi_acsnoop,
    output logic [2:0]        m_axi_acprot,
    output logic              m_axi_acvalid,
    input logic               m_axi_acready,
```

```

// Snoop response channel (CR)
input logic [4:0]          m_axi_crresp,
input logic               m_axi_crvalid,
output logic              m_axi_crready,

// Snoop data channel (CD)
input logic [DW-1:0]      m_axi_cddata,
input logic               m_axi_cdlast,
input logic               m_axi_cdvalid,
output logic              m_axi_cdready,

// Status outputs for clock gating
output logic              busy
);

```

6.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

6.3.3 CCU Interface (Input Side - FUB)

6.3.3.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
fub_acaddr	ADDR_WIDTH	Input	Snoop address
fub_acsnoop	4	Input	Snoop transaction type
fub_acprot	3	Input	Protection attributes
fub_acvalid	1	Input	Snoop address valid
fub_acready	1	Output	Snoop address ready

6.3.3.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
fub_crresp	5	Output	Snoop response

Port	Width	Direction	Description
fub_crvalid	1	Output	Snoop response valid
fub_crready	1	Input	Snoop response ready

6.3.3.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
fub_cddata	DATA_WIDTH	Output	Snoop data
fub_cdlast	1	Output	Last snoop data beat
fub_cdvalid	1	Output	Snoop data valid
fub_cdready	1	Input	Snoop data ready

6.3.4 Cache Responder Interface (Output Side - AXI)

6.3.4.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
m_axi_acaddr	ADDR_WIDTH	Output	Snoop address
m_axi_acsnoop	4	Output	Snoop transaction type
m_axi_acprot	3	Output	Protection attributes
m_axi_acvalid	1	Output	Snoop address valid
m_axi_acready	1	Input	Snoop address ready

6.3.4.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
m_axi_crresp	5	Input	Snoop response
m_axi_crvalid	1	Input	Snoop

Port	Width	Direction	Description
m_axi_crready	1	Output	response valid Snoop response ready

6.3.4.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
m_axi_cddata	DATA_WIDTH	Input	Snoop data
m_axi_cdlast	1	Input	Last snoop data beat
m_axi_cdvalid	1	Input	Snoop data valid
m_axi_cdready	1	Output	Snoop data ready

6.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

6.4 Functional Description

6.4.1 Triple Buffer Design

The module buffers all three snoop channels independently:

1. **AC Channel Buffer:** CCU → peer cache
2. **CR Channel Buffer:** Peer cache → CCU
3. **CD Channel Buffer:** Peer cache → CCU

6.4.2 Busy Signal Generation

```
assign busy = (int_ac_count > 0) || (int_cr_count > 0) ||
(int_cd_count > 0) ||
fub_acvalid || m_axi_crvalid || m_axi_cdvalid;
```

6.5 ACE Notes

- Snoop channels have no transaction ID; ordering is by channel only
- ACE requires CR and CD responses after the AC handshake, and both CR and CD must be returned in the same order as the AC addresses
- The module is a transport; it does not arbitrate or serialize snoops across multiple peer caches
- CRRESP bit semantics follow ACE: DataTransfer[0], Error[1], PassDirty[2], IsShared[3], WasUnique[4]

6.6 Related Modules

- **axi4ace_snoop_master_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_snoop_slave**: Complementary cache-side snoop responder
 - **axi4ace_snoop_monitor_lite**: ACE-aware monitor core for the snoop channels
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

6.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)

7 axi4ace_snoop_slave

ACE snoop-channel responder transport on the cache side: AC in from the coherency manager, CR/CD out back to the manager.

7.1 Overview

The `axi4ace_snoop_slave` module sits between the coherency manager (onyx) and a cache's snoop FSM (amber/jet). It receives a snoop command on the AC channel, buffers it, and presents it to the cache FSM on the FUB side. The cache FSM returns its response on CR and any data on CD; those are buffered back to the manager. It is a transport module and does not generate snoop responses or data.

7.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AC	int	2	Snoop address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CR	int	4	Snoop response channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CD	int	4	Snoop data channel skid buffer depth in entries (2..8 inclusive)
ADDR_WIDTH	int	32	Snoop address width
DATA_WIDTH	int	32	Snoop data width
AW	int	ADDR_WIDTH	Short alias for address width
DW	int	DATA_WIDTH	Short alias for data width
ACSize	int	AW + 4 + 3	Packed AC skid payload width
CRSize	int	5	Packed CR skid payload width
CDSize	int	DW + 1	Packed CD skid payload width

7.3 Ports

7.3.1 Module Declaration

```
module axi4ace_snoop_slave #(
    parameter int SKID_DEPTH_AC = 2,
    parameter int SKID_DEPTH_CR = 4,
    parameter int SKID_DEPTH_CD = 4,
    parameter int ADDR_WIDTH    = 32,
    parameter int DATA_WIDTH   = 32,
    parameter int AW            = ADDR_WIDTH,
    parameter int DW            = DATA_WIDTH,
    parameter int ACSIZE        = AW + 4 + 3,
    parameter int CRSIZE        = 5,
    parameter int CDSIZE        = DW + 1
) (
    // Global Clock and Reset
    input logic          aclk,
    input logic          aresetn,

    // Slave AXI Interface (Input Side) -- manager/coherency
    controller -> cache
    // Snoop address channel (AC)
    input logic [AW-1:0] m_axi_acaddr,
    input logic [3:0]    m_axi_acsnoop,
    input logic [2:0]    m_axi_acprot,
    input logic          m_axi_acvalid,
    output logic         m_axi_acready,

    // Snoop response channel (CR)
    output logic [4:0]   m_axi_crresp,
    output logic         m_axi_crvalid,
    input logic          m_axi_cready,

    // Snoop data channel (CD)
    output logic [DW-1:0] m_axi_cddata,
    output logic         m_axi_cdlast,
    output logic         m_axi_cdvalid,
    input logic          m_axi_cdready,

    // Fabric/Upstream Buffer Interface (Output Side) -- cache snoop
    FSM
    // Snoop address channel (AC)
    output logic [AW-1:0] fub_acaddr,
    output logic [3:0]    fub_acsnoop,
    output logic [2:0]    fub_acprot,
    output logic         fub_acvalid,
```

```

    input logic                                fub_acready,

    // Snoop response channel (CR)
    input logic [4:0]                          fub_crresp,
    input logic                                fub_crvalid,
    output logic                               fub_crready,

    // Snoop data channel (CD)
    input logic [DW-1:0]                      fub_cddata,
    input logic                                fub_cdlast,
    input logic                                fub_cdvalid,
    output logic                               fub_cdready,

    // Status outputs for clock gating
    output logic                               busy
);

```

7.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

7.3.3 Manager Interface (Input Side - AXI)

7.3.3.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
m_axi_acaddr	ADDR_WIDTH	Input	Snoop address
m_axi_acsnoop	4	Input	Snoop transaction type
m_axi_acprot	3	Input	Protection attributes
m_axi_acvalid	1	Input	Snoop address valid
m_axi_acready	1	Output	Snoop address ready

7.3.3.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
m_axi_crresp	5	Output	Snoop response
m_axi_crvalid	1	Output	Snoop response valid
m_axi_crready	1	Input	Snoop response ready

7.3.3.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
m_axi_cddata	DATA_WIDTH	Output	Snoop data
m_axi_cdlast	1	Output	Last snoop data beat
m_axi_cdvalid	1	Output	Snoop data valid
m_axi_cdready	1	Input	Snoop data ready

7.3.4 Cache FSM Interface (Output Side - FUB)

7.3.4.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
fub_acaddr	ADDR_WIDTH	Output	Snoop address
fub_acsnoop	4	Output	Snoop transaction type
fub_acprot	3	Output	Protection attributes
fub_acvalid	1	Output	Snoop address valid
fub_acready	1	Input	Snoop address ready

7.3.4.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
fub_crresp	5	Input	Snoop response
fub_crvalid	1	Input	Snoop response valid
fub_crready	1	Output	Snoop response ready

7.3.4.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
fub_cddata	DATA_WIDTH	Input	Snoop data
fub_cdlast	1	Input	Last snoop data beat
fub_cdvalid	1	Input	Snoop data valid
fub_cdready	1	Output	Snoop data ready

7.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

7.4 Functional Description

7.4.1 Triple Buffer Design

The module buffers all three snoop channels independently:

1. **AC Channel Buffer:** Manager → cache FSM
2. **CR Channel Buffer:** Cache FSM → manager
3. **CD Channel Buffer:** Cache FSM → manager

7.4.2 Busy Signal Generation

```
assign busy = (int_ac_count > 0) || (int_cr_count > 0) ||  
(int_cd_count > 0) ||  
             m_axi_acvalid || fub_crvalid || fub_cdvalid;
```

7.5 ACE Notes

- Snoop channels have no transaction ID; ordering is by channel only
- ACE requires CR and CD responses after the AC handshake, and both CR and CD must be returned in the same order as the AC addresses
- The module does not enforce the stricter cache-ip convention (complete CD before CR); that ordering is produced by the cache FSM driving the FUB side
- CRRESP bit semantics follow ACE: DataTransfer[0], Error[1], PassDirty[2], IsShared[3], WasUnique[4]

7.6 Related Modules

- **axi4ace_snoop_slave_monlite**: Lite-monitor wrapper for this core
 - **axi4ace_snoop_master**: Complementary CCU-side snoop initiator
 - **axi4ace_snoop_monitor_lite**: ACE-aware monitor core for the snoop channels
 - **gaxi_skid_buffer**: Underlying buffer infrastructure
-

7.7 Navigation

- [← Back to ACE Index](#)
- [← Back to rtl-amba Index](#)
- [← Back to Main Documentation Index](#)